# Shiv Nadar University Chennai

| Name of Student | **KR Srinidhi Rajamane** | Reg. No. **24011101059** |
|---|---|---|
| Degree & Branch | B.Tech Artificial Intelligence & Data Science Section A | Semester V |
| Subject Code & Name | CS3807 – Deep Learning Laboratory | AY: 2026–27 |

## Experiment 1
### Implementation of a Single Layer Perceptron for Binary Classification

## 1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

## 2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

## 3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

### Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$ : Input vector
- $\mathbf{w}$ : Weight vector
- $b$ : Bias
- $z$ : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where $\eta$ denotes the learning rate.

## 4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

### Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

| Activation | Output Range | Typical Applications |
|---|---|---|
| Step | $\{0, 1\}$ | Classical Perceptron |
| Sigmoid | $(0, 1)$ | Binary Classification Output Layer |
| Tanh | $(-1, 1)$ | Hidden Layers (Traditional Networks) |
| ReLU | $[0, \infty)$ | Hidden Layers in CNNs and MLPs |
| Leaky ReLU | $(-\infty, \infty)$ | Avoiding Dying ReLU |
| Softmax | $(0, 1)$ | Multi-class Classification Output Layer |

**Note:** This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

## 5. Dataset Description

**Dataset:** Banknote Authentication Dataset

**Source:** UCI Machine Learning Repository

**Download Link:**

https://archive.ics.uci.edu/dataset/267/banknote+authentication

| | |
|---|---|
| Instances | 1372 |
| Features | 4 Numerical Features |
| Classes | 2 |
| Missing Values | None |
| Task | Binary Classification |

**Feature Description**

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

## 6. Experimental Procedure

**Task 1: Dataset Exploration**

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

**Expected Output**
Dataset summary and statistical information.

**Task 2: Exploratory Data Analysis**
Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

**Task 3: Data Preprocessing**

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

**Task 4: Perceptron Implementation**
Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

**Task 5: Model Training**
Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

**Task 6: Model Evaluation**

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

## 7. Source Code

The complete implementation was carried out in Python using NumPy, Pandas, Matplotlib, Seaborn and Scikit-learn. The full, executable source code (organized task-wise, exactly as it was run) is available on GitHub:

**https://github.com/Srinidhi-Krishna/Deep-Learning-Lab/blob/main/Lab_1/DL_LAB_1.ipynb**

# 8. Generated Plots with Interpretation

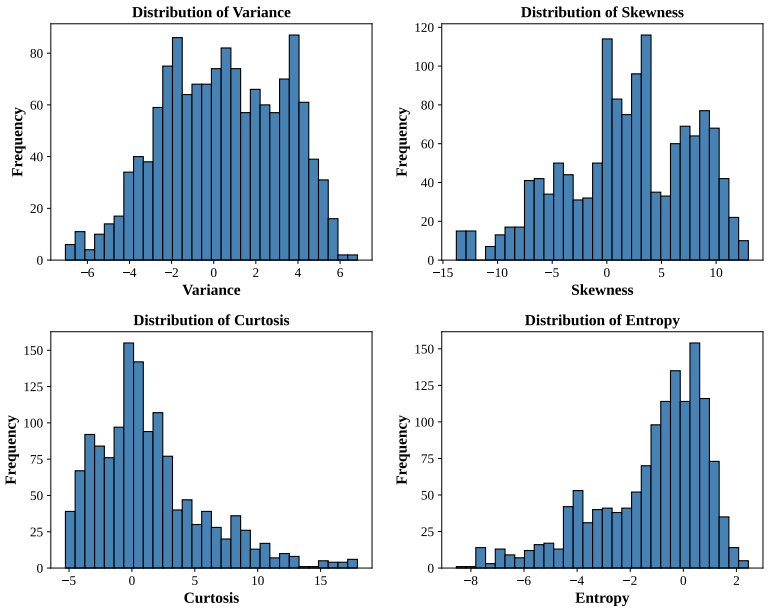


**Figure 1:** Frequency distribution histograms of Variance, Skewness, Curtosis and Entropy.

**Interpretation:** This plot gives four histograms each one depicting Variance, Skewness, Curtosis and Entropy. We can infer from these plots that not of these distributions appear bell-shaped. Skewness and Curtosis have longer tails on one side and the Entropy is has more negative values.

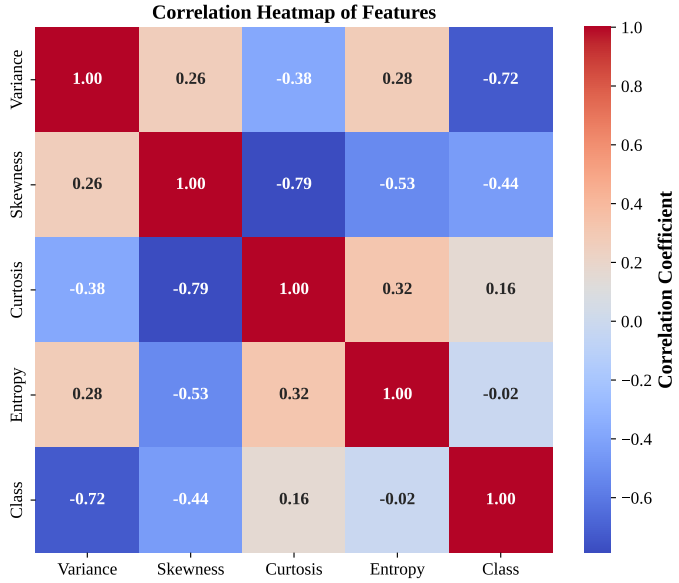


**Figure 2:** Pearson correlation heatmap among the four features and the class label.

**Interpretation:**This plot tells us that Variance has the strongest correlation to Class while Variance, Skew-

ness and Curtosis all have some negative weights. Entropy is around the zero quite often. Variance and Curtosis are also correlated to each other so there is some redundancy but not enough to make any of the feature useless.

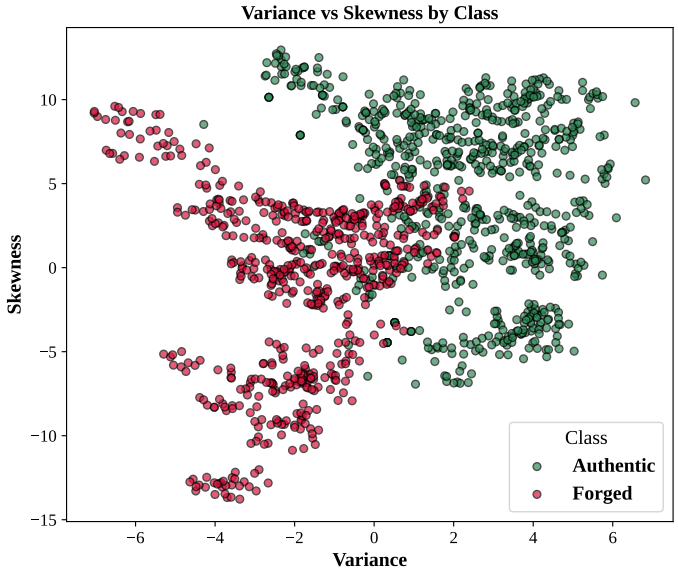


**Figure 3:** Scatter plot of Variance versus Skewness, colour-coded by class.

**Interpretation:** This plot gives a scatter plot of variance vs skewness for two classes which are authentic and forged.Since there is some amount of overlap the single layer perceptron doesn't work very well here since there two classes are not lineraly separable completely.

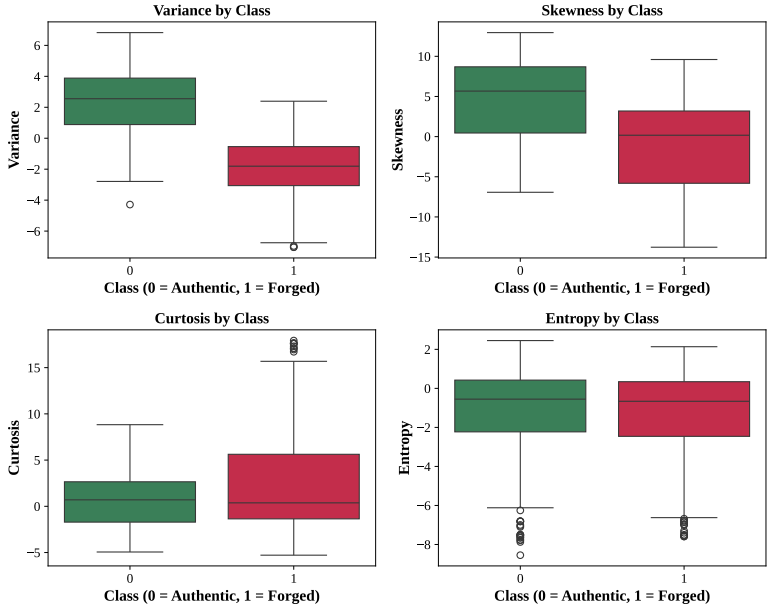


**Figure 4:** Boxplots of each feature grouped by class.

**Interpretation:** This plot gives the scatter plot for Variance, Skewness, Curtosis and Entropy. Variance's boxes does noit not overlap which explains why it has the highest final weight whereas Entropy's boxes almost overlap completely which explains why it is having the smallest final weight.

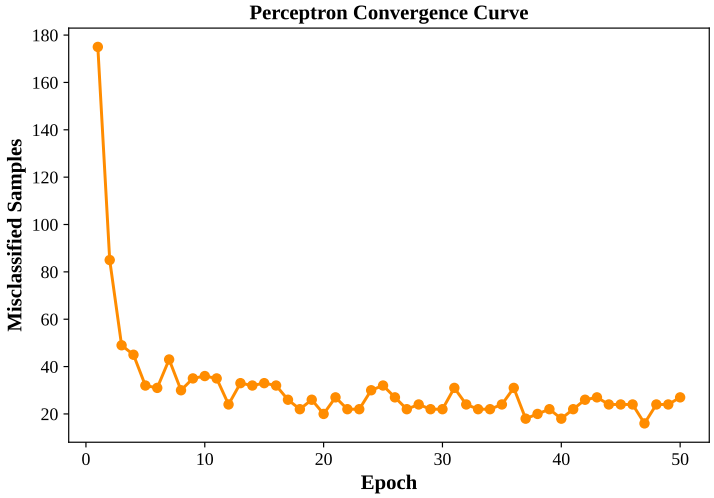


**Figure 5:** Misclassified training samples per epoch (learning rate 0.01).

**Interpretation:** This plot gives the misclassified training samples per epoch. As we can observe the Errors drop fast for the first 8 to 10 epochs then flattens about 16 and 30 for the rest of the training. It never reaches zero which tells us that the data is not completely linearly separable as if they were the algorithm would eventually find a zero-error boundary.

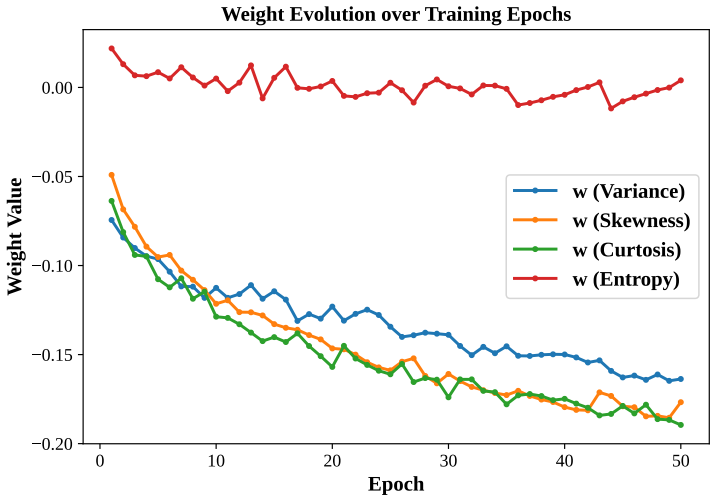


**Figure 6:** Evolution of the four feature weights across training epochs.

**Interpretation:** This plot gives the evolution of all the four weights Variance, Skewness ,Curtosis and Entropy across training epochs. Here Variance, Skewness and Curtosi ssettle near similar negative values while Entropy stays close to zero the whole time. Thus Entropy is basically the weakest feature here.

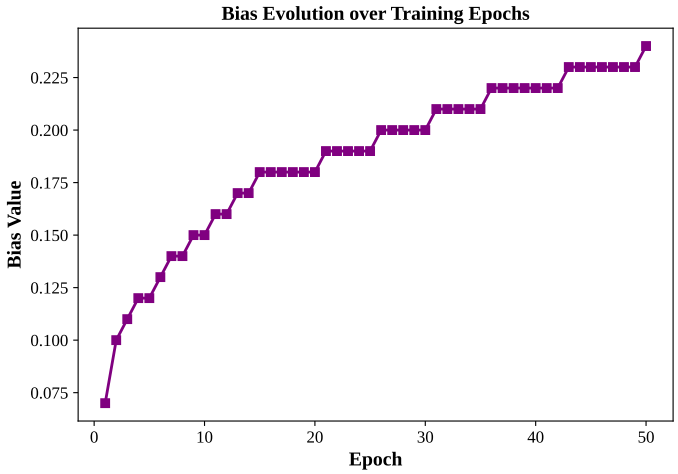


**Figure 7:** Evolution of the bias term across training epochs.

**Interpretation:** This plot gives the evolution of bias term across training epochs. The bias climbs in a rough pattern which means the model is still making corrections right up to the end.

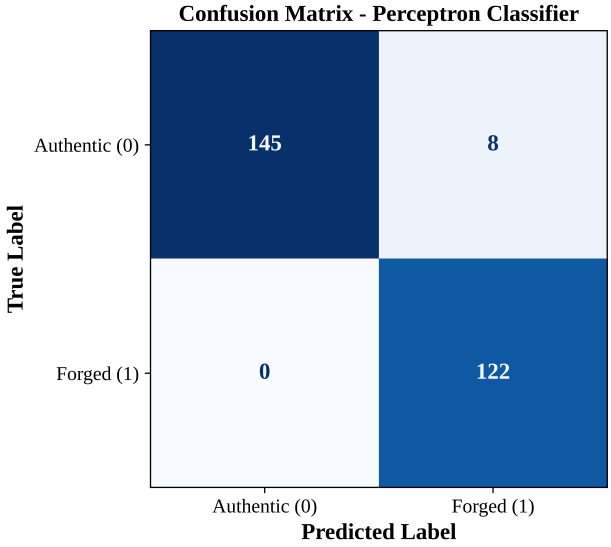


**Figure 8:** Confusion matrix on the held-out test set (275 samples).

**Interpretation:** This plot gives the confusion matrix were Only 8 out of 275 test notes were misclassified.For currency verification it is a good error pattern since a false alarm on a genuine note is a lot less costly than letting a fake note pass.

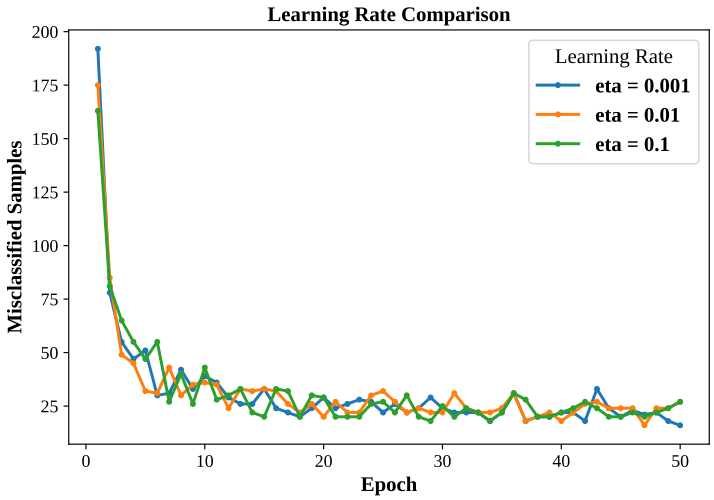


**Figure 9:** Convergence behaviour of the perceptron under three different learning rates.

**Interpretation:** This plot gives the convergence behaviour of the perceptron under different learning rates. All three curves drop rapidly at first but settle at different noise levels. This shows that the model is not very dependent on learning rates

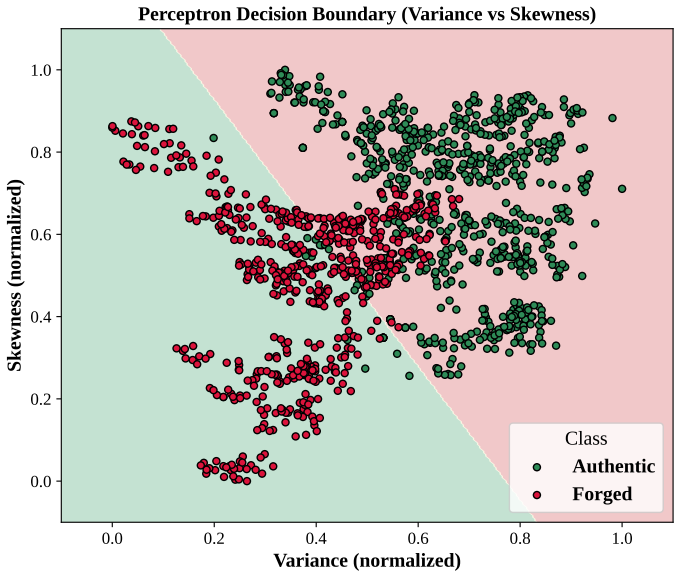


**Figure 10:** Decision boundary learned by a 2-feature perceptron (Variance vs Skewness).

**Interpretation:** This plot gives the decision boundary that shows how the two classes are separated. The overlaps are the reason for the misclassifications.

## 9. Mandatory Plots

| Plot | Purpose | Expected Inference |
|---|---|---|
| Feature Histograms | Distribution Analysis | Identify skewness and outliers |
| Correlation Heatmap | Feature Relationship | Observe highly correlated features |
| Scatter Plot | Class Distribution | Determine whether classes are approximately linearly separable |
| Training Error vs Epoch | Learning Behaviour | Verify convergence of the perceptron |
| Weight Evolution | Parameter Analysis | Observe stabilization of learned weights |
| Bias Evolution | Parameter Analysis | Understand the role of the bias term |
| Confusion Matrix | Performance Evaluation | Interpret TP, FP, TN and FN |
| Learning Rate Comparison | Hyperparameter Study | Compare convergence behaviour for different learning rates |
| Decision Boundary (Optional) | Visualization | Illustrate the separating hyperplane using two selected features |

# 10. Performance Tables

**Training Summary**

| | |
|---|---|
| Dataset Size | 1372 samples |
| Train/Test Split | 80% / 20% (1097 train / 275 test) |
| Learning Rate | 0.01 |
| Epochs | 50 (ran for the full budget; did not reach zero error) |
| Final Weights | Variance: $-0.1637$, Skewness: $-0.1767$, Curtosis: $-0.1895$, Entropy: 0.0040 |
| Final Bias | 0.2400 |
| Accuracy | 0.9709 (97.09%) |
| Precision | 0.9385 (93.85%) |
| Recall | 1.0000 (100.00%) |
| F1-score | 0.9683 (96.83%) |

**Epoch-wise Learning**

*(Weight 1 = Variance, Weight 2 = Skewness; selected epochs shown for brevity)*

| Epoch | Errors | Weight 1 | Weight 2 | Bias |
|---|---|---|---|---|
| 1 | 175 | −0.0744 | −0.0491 | 0.0700 |
| 5 | 32 | −0.0964 | −0.0952 | 0.1200 |
| 10 | 36 | −0.1125 | −0.1215 | 0.1500 |
| 20 | 20 | −0.1231 | −0.1465 | 0.1800 |
| 30 | 22 | −0.1389 | −0.1609 | 0.2000 |
| 40 | 18 | −0.1499 | −0.1795 | 0.2200 |
| 47 | 16 | −0.1642 | −0.1847 | 0.2300 |
| 50 | 27 | −0.1637 | −0.1767 | 0.2400 |

# 11. Experimental Results

## 11.1 Dataset Exploration (Task 1)

The dataset loaded fine – 1372 rows and 5 columns (4 features + class label). `df.isnull().sum()` showed zero missing values anywhere, so no imputation needed. Class split is 762 authentic vs 610 forged, fairly balanced, no resampling required. From `df.describe()`, Skewness has the widest spread (std $\approx 5.87$, range roughly $-13.77$ to 12.95) while Entropy is the most tightly packed (std $\approx 2.10$). That gap in scale is basically the reason normalization was needed before training.

## 11.2 Data Preprocessing (Task 3)

After `MinMaxScaler`, all four features got squeezed into $[0,1]$ (checked the printed min/max: 0.000 and 1.000). Stratified 80/20 split gave 1097 train / 275 test samples, class ratio preserved on both sides.

## 11.3 Model Training (Task 5)

Trained for a fixed 50 epochs, $\eta = 0.01$. Epoch 1 started with 175 errors, dropped fast to around 30–40 by epoch 10, then just kept bouncing between roughly 16 and 30 for the rest of training instead of settling down. Lowest point was epoch 47 with 16 errors, but it climbed again right after. Along with the epoch table in Section 8, this basically confirms the four features together aren't perfectly linearly separable – the perceptron never locks onto a zero-error line in 50 epochs, it just keeps circling a decent solution.

## 11.4 Model Evaluation (Task 6)

Final-epoch weights on the test set gave:

| Metric | Value |
|---|---|
| Accuracy | 0.9709 (97.09%) |
| Precision | 0.9385 (93.85%) |
| Recall | 1.0000 (100.00%) |
| F1-score | 0.9683 (96.83%) |

Confusion matrix on the 275 test samples:

$$\begin{bmatrix} 145 & 8 \\ 0 & 122 \end{bmatrix}$$

145 authentic notes correctly caught, 8 authentic notes wrongly flagged as forged, and zero forged notes slipped through as authentic. That's why recall on the forged class is a perfect 1.0 – for something like currency checking, that's the error type you actually want to avoid.

### 11.5 Effect of Learning Rate (Task 7)

Tried three rates, each trained fresh for 50 epochs:

| Learning Rate ($\eta$) | Epochs Run | Final Misclassified |
|---|---|---|
| 0.001 | 50 | 16 |
| 0.01 | 50 | 27 |
| 0.1 | 50 | 27 |

Bit surprising – I expected the smallest rate to be the slowest one, but $\eta = 0.001$ actually ends up with the fewest errors. Small steps mean the weight vector moves cautiously and doesn't overshoot, while $\eta = 0.01$ and 0.1 take bigger corrective jumps and keep bouncing the boundary around instead of settling, which is a known thing with the perceptron rule on data that isn't perfectly separable.

### 11.6 Comparison with Scikit-learn (Task 9, Optional)

Ran scikit-learn's `Perceptron` with the same settings (`max_iter=50`, `eta0=0.01`) on the same split and got identical numbers to the from-scratch version – 0.9709 accuracy, 0.9385 precision, 1.0000 recall, 0.9683 F1 for both. Good sign that the weight update, step activation and forward pass were coded correctly.

# 12. Additional Tasks

## 12.1 Step vs. Sigmoid Activation

| **Property** | **Step** | **Sigmoid** |
|---|---|---|
| Output | Hard $\{0, 1\}$ | Smooth $(0, 1)$ |
| Derivative | Zero everywhere, undefined at $z = 0$ | Well-defined, non-zero almost everywhere |
| Gradient flow | None – blocks backpropagation | Enables gradient-based learning |
| Confidence info | None (hard decision only) | Yes, output is a probability-like score |

The Step function is not used in modern deep learning because it is non-differentiable and thus gradient descent and backpropagation cannot flow a useful error signal back through it. Sigmoid also has the advantage of giving a soft and interpretable output whereas Step only ever gives a hard class label.

## 12.2 Comparison with Scikit-learn's Perceptron

As reported in Section 11.6, running `sklearn.linear_model.Perceptron` with 50 iterations and 0.01 learning rate on the identical train/test split produced 97.09% accuracy,93.85% precision, 100%recall and 96.83%F1 scores that exactly matched the from-scratch implementation .

## 12.3 Effect of Learning Rate

The larger step sizes overshoot the decision boundary on data that is not perfectly linearly separable instead of settling into it.

## 12.4 Why a Single Layer Perceptron Cannot Solve XOR

XOR is not linearly separable. Since a single-layer perceptron can only represent one linear decision boundary, it is mathematically incapable of solving XOR.

## 12.5 Effect of Feature Normalization on Convergence

Convergence speed and stability improve significantly with feature normalization. Since all features are brought to similar range, dominance is prevented by high magnitude features.

# 13. Discussion

The from-scratch implemented perceptron performs well with a test accuracy of 97.09% on the Banknote Authentication dataset. This shows that the classes are almost, but not fully, linearly separable. Although the

model had misclassifications, it never allowed a forged note to pass through. Overall the model performed excellently.

## 14. Conclusion

This experiment therefore successfully demonstrates a Single Layer Perceptron. While the model is highly efficient for datasets that are approximately linearly separable by achieving over 97% accuracy it fails while dealing with highly overlapping data. Thus transition to multilayer networks and differentiable activation functions is necessary to deal with more complex and non-linear challenges.

## 15. References


1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.